

Rapport de Laboratoire : Introduction à Spring Boot & Spring MVC

Application Web de Conversion de Température en Kotlin

Étudiant(e) :	Manasse Biaya	Numéro d'étudiant :	300421324
Cours :	SEG 3102 / 3502	Dépôt GitHub :	https://github.com/mxnsbiaya/springBootIntro
Date de soumission :	Septembre 2026	Statut du projet :	✓ 100% Fonctionnel & Testé

1. Brève description du travail réalisé

Ce laboratoire consiste en une initiation pratique à l'architecture logicielle offerte par le framework **Spring Boot 3** et son module **Spring MVC** en utilisant le langage moderne **Kotlin**. L'objectif central est la mise en place d'une application web MVC complète permettant la conversion bidirectionnelle de températures entre les degrés Celsius et Fahrenheit, avec validation des données et affichage dynamique des vues.

Fonctionnalités et Architecture implémentées :

- **Architecture MVC (Modèle - Vue - Contrôleur)** : Séparation nette entre la logique de contrôle (`WebController`), les données échangées (`Model` Spring) et la couche de présentation (Thymeleaf).
- **Contrôleur de requêtes HTTP (`WebController.kt`)** :
 - Initialisation globale des attributs via `@ModelAttribute` (attributs `celsius`, `fahrenheit` et `error`).
 - Routage de la racine `/` renvoyant la vue d'accueil `home`.
 - Gestionnaire d'action `/convert` (méthode GET) qui intercepte les paramètres de formulaire (`celsius`, `fahrenheit`, `operation`), applique la formule de conversion appropriée et injecte les résultats formatés (`String.format("%.2f", ...)`) dans le modèle.
- **Gestion robuste des exceptions et cas d'erreur** : Interception des `NumberFormatException` pour détecter les saisies invalides (chaînes non numériques, champs vides incorrects) et association des identifiants d'erreur correspondants (`CelsiusFormatError`, `FahrenheitFormatError`, `OperationFormatError`).
- **Vues dynamiques avec Thymeleaf (`home.html`)** : Moteur de template intégrant un formulaire HTML, la liaison bidirectionnelle de valeurs (`th:value`) et un affichage conditionnel des messages d'erreur (`th:switch` et `th:case`).
- **Suite de tests automatisés (`WebControllerTest.kt`)** : Conception et validation d'une batterie complète de tests avec `MockMvc` vérifiant l'affichage de la page d'accueil, les calculs de conversion pour différentes valeurs (notamment 0°C = 32°F et 100°C = 212°F), ainsi que la détection correcte de toutes les erreurs de validation.

Environnement d'exécution : Le projet a été configuré et validé avec succès avec **Java 21 LTS** (OpenJDK Temurin 21.0.6+7), **Gradle 8.5**, **Spring Boot 3.4.5** et **Kotlin 1.9.25**.

2. Étapes nécessaires pour installer et exécuter le projet

Pour reproduire et exécuter le projet dans n'importe quel environnement de développement, suivre les étapes ci-dessous :

1. Prérequis :

- **Java Development Kit (JDK) 21** installé sur le système.
- **Git** pour le contrôle de version.

2. Cloner le dépôt de code :

```
git clone https://github.com/stephanesome/springBootIntro.git
cd springBootIntro
```

3. Définir JAVA_HOME (vers le JDK 21) :

```
# Sous Windows (PowerShell) :
$env:JAVA_HOME = "C:\Program Files\Eclipse Adoptium\jdk-21" # ou votre chemin JDK 21
$env:Path = "$env:JAVA_HOME\bin;" + $env:Path

# Sous Linux / macOS (Bash) :
export JAVA_HOME=/path/to/jdk-21
export PATH=$JAVA_HOME/bin:$PATH
```

4. Lancer les tests automatisés :

```
# Windows :
.\gradlew.bat test

# Linux / macOS :
./gradlew test
```

5. Démarrer le serveur d'application :

```
# Windows :
.\gradlew.bat bootRun

# Linux / macOS :
./gradlew bootRun
```

6. Accéder à l'application web : Ouvrir un navigateur web moderne à l'adresse locale : `http://localhost:8080/`.

3. Captures d'écran de l'application et démonstration

Les captures d'écran ci-dessous attestent du bon fonctionnement de chaque composant de l'application :

Temperature Converter

Celsius:

Fahrenheit:

Figure 1 : Page d'accueil initiale (`http://localhost:8080/`). Formulaire prêt pour la saisie des températures avec boutons de conversion.

Temperature Converter

Celsius:

Fahrenheit:

Figure 2 : Conversion réussie Celsius → Fahrenheit. Saisie : `100 °C`, Résultat calculé : `212.00 °F`.

Temperature Converter

Celsius:

Fahrenheit:

Figure 3 : Conversion réussie Fahrenheit → Celsius. Saisie : `68 °F`, Résultat calculé : `20.00 °C`.

Temperature Converter

Wrong value provided for Celsius - must be a number

Celsius:

Fahrenheit:

Figure 4 : Gestion d'erreur - Saisie invalide pour Celsius (`"abc"`). Message rouge d'avertissement affiché par Thymeleaf.

Temperature Converter

Wrong value provided for Fahrenheit - must be a number

Celsius:

Fahrenheit:

Figure 5 : Gestion d'erreur - Saisie invalide pour Fahrenheit (`"xyz"`). Détection de `NumberFormatException` et affichage de l'erreur.

Test Summary

9	0	0	0.898s	100%
tests	failures	ignored	duration	successful

Packages		Classes				
Package	Tests	Failures	Ignored	Duration	Success rate	
seg3x02.converter	9	0	0	0.898s	100%	

Generated by Gradle 8.5 at Sep 29, 2026, 9:47:02 PM

Figure 6 : Rapport de tests Gradle HTML (`build/reports/tests/test/index.html`) : 100% de réussite sur les 9 tests exécutés en 0.898s.

4. Résultats des tests automatisés

La suite de tests unitaires et d'intégration Spring MVC (`@WebMvcTest`) a été exécutée avec succès via le moteur JUnit 5.

Classe de Test	Méthode de Test	Objectif vérifié	Résultat
<code>ConverterApplicationTests</code>	<code>contextLoads()</code>	Chargement correct du contexte Spring Boot	RÉUSSI
<code>WebControllerTest</code>	<code>request_to_home()</code>	GET / retourne statut 200 et vue "home"	RÉUSSI
<code>WebControllerTest</code>	<code>celsius_to_fahrenheit_conversion()</code>	Conversion 0 °C → 32.00 °F	RÉUSSI
<code>WebControllerTest</code>	<code>celsius_to_fahrenheit_boiling_point()</code>	Conversion 100 °C → 212.00 °F	RÉUSSI
<code>WebControllerTest</code>	<code>fahrenheit_to_celsius_conversion()</code>	Conversion 32 °F → 0.00 °C	RÉUSSI
<code>WebControllerTest</code>	<code>fahrenheit_to_celsius_boiling_point()</code>	Conversion 212 °F → 100.00 °C	RÉUSSI
<code>WebControllerTest</code>	<code>celsius_format_error()</code>	Saisie 'abc' pour Celsius produit CelsiusFormatError	RÉUSSI
<code>WebControllerTest</code>	<code>fahrenheit_format_error()</code>	Saisie 'xyz' pour Fahrenheit produit FahrenheitFormatError	RÉUSSI
<code>WebControllerTest</code>	<code>invalid_operation_error()</code>	Opération inconnue produit OperationFormatError	RÉUSSI

5. Instructions pour la remise sur Brightspace

1. Créer un dépôt GitHub public (ou privé avec accès accordé au professeur) nommé par exemple `springBootIntro`.

2. Pousser le code vers votre dépôt :

```
git remote set-url origin https://github.com/[VotreNomUtilisateur]/springBootIntro.git
git add .
git commit -m "Laboratoire finalisé avec tests complets et documentation"
git push -u origin master
```

3. Télécharger ce rapport PDF généré (ou le convertir après avoir rempli vos nom et numéro étudiant).

4. Déposer sur le devoir Brightspace :

- Le **lien vers votre dépôt GitHub**
- Le **fichier PDF** du présent rapport